

JavaScript-07

OOP i JavaScript

24 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg

Book

methods

buy

read

review

info

t Objects

javascript keyword

property name

method name



För en kanin

OOP i JavaScript

När man skriver kod i JavaScript börjar man ofta med lösa funktioner och variabler, det som också brukar kallas för procedurrell programmering. Det är ett alldeles utmärkt sätt att skriva kod när man har korta skript som inte ska göra särskilt mycket. Ska man däremot skriva lite större system som till exempel ett spel, en admin-panel eller en webbshop, blir det snabbt rörigt och ostrukturerat att skriva på det sättet. Objektorienterad programmering (OOP) handlar om att samla ihop relaterade variabler (data) och funktioner (beteenden) på ett och samma ställe. Detta ställe kallar vi för ett Objekt.

Ett objekt

Ett sätt att skapa objekt i JavaScript är med hjälp av måsvingar { och }. Det kallas för att skapa en objektliteral. Literal betyder ungefär ett fast värde eller ett utskrivet värde. Det kan se ut som i bild 1 nedan.

```
const bil = {  
  marke: "Volvo", // Egenskap  
  farg: "röd", // Egenskap  
  hastighet: 0, // Egenskap  
  
  // Metod (en funktion inuti ett objekt)  
  gasa() {  
    this.hastighet += 10;  
    console.log(`Bilen, som är ${this.farg}, kör nu i ${this.hastighet} km/h.`);  
  }  
};  
  
bil.gasa(); // Utskrift: Bilen, som är röd, kör nu i 10 km/h.
```

Bild 1 En objektliteral.

I ett objekt kallas variablerna för egenskaper (properties) och funktionerna kallas för metoder. Objektet bil har både properties och en metod.

Flera objekt

Om man bara behöver ett eller några få objekt, fungerar det alldeles utmärkt att skapa objekten, ett och ett. Tänk dig däremot om du gör ett bilspel med 100 olika bilar. Då är det inte lika roligt längre. Det är här klasser kommer in. Det heter class på JavaScript. Se på en klass som en ritning eller en kakform. Den bestämmer hur objekten ska vara utformade. Man kan återanvända ritningen eller kakformen för att göra nya objekt. En klass motsvarande objektet enligt bild 1 skulle då se ut enligt bild 2.

```
class Bil {  
  // Konstruktor körs ALLTID automatiskt när vi skapar en ny bil  
  constructor(märke, färg) {  
    this.märke = märke;    // Sätter egenskapen utifrån vad vi skickar in  
    this.färg = färg;  
    this.hastighet = 0;    // Alla nya bilar startar på 0 km/h  
  }  
  
  gasa() {  
    this.hastighet += 20;  
    console.log(`${this.märke} gasar! Ny hastighet: ${this.hastighet} km/h.`);  
  }  
}  
  
// Vi skapar två olika instanser från samma klass med nyckelordet "new"  
const bil1 = new Bil("Volvo", "Röd");  
const bil2 = new Bil("Saab", "Blå");  
  
bil1.gasa(); // Utskrift: Volvo gasar! Ny hastighet: 20 km/h.  
bil2.gasa(); // Utskrift: Saab gasar! Ny hastighet: 20 km/h.
```

Bild 2 Klassen Bil, med flerar properties och metoden gasa().

Längst ner i bild 2 ser vi att det skapas två olika bilar, genom att skicka med märke och färg till konstruktorn för klassen.

Arv (Inheritance)

Arv innebär att man kan återanvända koden. Ibland vill vi skapa en ny klass som påminner mycket om en klass vi redan har, men med några extra funktioner. Istället för att kopiera all kod kan vi använda arv med nyckelordet `extends`. Den nya klassen (underklassen) ärver då automatiskt alla egenskaper och metoder från den gamla klassen (superklassen).

```
// Superklass (Förälder)
class Fordon {
  constructor(namn) {
    this.namn = namn;
  }
  tuta() {
    console.log("Tuuut!");
  }
}

// Underklass (Barn) som ärver från Fordon
class Elbil extends Fordon {
  constructor(namn, batteriNivå) {
    // super() anropar förälderns constructor så att egenskapen 'namn' sätts
    super(namn);
    this.batteriNivå = batteriNivå;
  }

  visaBatteri() {
    console.log(`${this.namn} har ${this.batteriNivå}% batteri kvar.`);
  }
}

const minTesla = new Elbil("Tesla Model 3", 85);
minTesla.tuta();           // Utskrift: Tuuut! (Ärvt metod från Fordon)
minTesla.visaBatteri();    // Utskrift: Tesla Model 3 har 85% batteri kvar.
```

Bild 3 Användning av `extends` and `super()`

Inkapsling (Encapsulation)

Inom OOP vill man ofta låsa vissa variabler inuti klassen så att ingen kan gå in utifrån och ändra på dem av misstag (t.ex. sätta ett bankkonto-saldo till en miljon eller ändra en bils hastighet utan att gasa). I modern JavaScript gör vi en egenskap eller metod privat genom att sätta ett fyrkantstecken (#) framför namnet. Då går det bara att läsa eller ändra det värdet inifrån klassens egna metoder.

```
class BankKonto {
  #saldo = 0; // Det här är nu en PRIVAT variabel tack vare #

  sättInPengar(kronor) {
    if (kronor > 0) {
      this.#saldo += kronor;
      console.log(`Satte in ${kronor} kr.`);
    }
  }

  visaSaldo() {
    console.log(`Ditt saldo är: ${this.#saldo} kr.`);
  }
}

const mittKonto = new BankKonto();
mittKonto.sättInPengar(500);

// Det här kommer att krascha eller ge ett felmeddelande i konsolen:
// mittKonto.#saldo = 1000000;

mittKonto.visaSaldo(); // Utskrift: Ditt saldo är: 500 kr.
```

Bild 4 Användning av # som privat medlemsvariabel.

Getters och Setters

Ibland vill vi tillåta att man läser eller ändrar en egenskap utifrån, men vi vill lägga till en säkerhetskontroll eller omvandling på vägen. För detta använder vi nyckelorden `get` (hämta) och `set` (ändra). Det fiffiga med getters och setters är att de ser ut och används som vanliga egenskaper när man anropar dem (utan parenteser!), men under huven körs de som dolda funktioner.

```
class Användare {
  constructor(namn) {
    this._namn = namn; // Vi använder understreck (_), för att variabeln är "intern"
  }

  // En GETTER för att hämta namnet med stor begynnelsebokstav
  get namn() {
    return this._namn.toUpperCase();
  }

  // En SETTER för att kontrollera namnet innan det sparas
  set namn(nyttNamn) {
    if (nyttNamn.length > 2) {
      this._namn = nyttNamn;
    } else {
      console.log("Fel: Namnet måste vara längre än 2 bokstäver!");
    }
  }
}

const person = new Användare("kalle");

console.log(person.namn); // Utskrift: KALLE (Getter anropas utan parenteser!)

person.namn = "Bo";      // Utskrift: Fel: Namnet måste vara längre än 2 bokstäver!
person.namn = "Erik";    // Ändringen godkänns
```

Bild 5: Getters och setters fungerar som dörrvakter.